PYTHON KURSU • HAFTA 7 • GÜN 2

SQLite 1: Veri Tabanına Giriş

Ders 14 / 32 • 3 saat • VS Code + Python 3.12

Bu dersin sonunda…

- Veri tabanı, tablo, satır, sütun ve birincil anahtar kavramlarını açıklayabileceksiniz.
- sqlite3 ile bir veri tabanı dosyasına bağlanıp tablo oluşturabileceksiniz.
- INSERT ile güvenli biçimde (? yer tutucularıyla) kayıt ekleyebileceksiniz.
- SELECT, ORDER BY, LIMIT ve COUNT/AVG ile kayıtları okuyabileceksiniz.
- Veri tabanı dosyasını DB Browser for SQLite ile inceleyebileceksiniz.

AÇILIŞ • 10 DK

Bugünün zaman planı

Saat	Süre	Bölüm
0:00–0:10	10 dk	Açılış: hedefler, Ders 13'in tekrarı
0:10–1:25	75 dk	Blok 1: veri tabanı ve SQL kavramları • SQLite ve bağlantı • CREATE TABLE • INSERT ve ? yer tutucuları • commit
1:25–1:40	15 dk	Mola
1:40–2:55	75 dk	Blok 2: SELECT, ORDER BY, LIMIT • COUNT/AVG • DB Browser for SQLite • kütüphane ve harcama uygulamaları • alıştırmalar
2:55–3:00	5 dk	Ödev ve gelecek ders

AÇILIŞ • DERS 13'IN TEKRARI • 6 DK

Dört soruda geçen ders

SORU 1 • Ne fark eder?

```
with st.form("f"):
    ad = st.text_input("Ad")
    st.form_submit_button("Gönder")
```

SORU 2 • Hangisi?

```
# Yapay zekâ modelini bir kez yüklemek için
@st.cache_data  mı?
@st.cache_resource  mı?
```

SORU 3 • Sunucu bunu nereden bilir?

```
ModuleNotFoundError:
No module named 'transformers'
```

SORU 4 • Nerede durmalı?

```
API_ANAHTARI = "abc123"
# app.py içinde mi, başka yerde mi?
```

AÇILIŞ • ÖDEV KONTROLÜ • 4 DK

Ödev 1: model nerede yanıldı?

Cümle türü	Örnek	Neden zor?
İroni	“Harika, kargo yine bir hafta sürdü.”	olumlu kelime, olumsuz anlam
Karışık	“Kumaşı ince ama rengi güzel.”	iki duygu bir arada
Olumsuzlama	“Fena değil, beğenmedim diyemem.”	çifte olumsuzluk
Tarafsız	“Paket salı günü teslim edildi.”	modelde nötr sınıf yok

Gerçek sonuçları birlikte okuyun: model hangilerinde yanıldı, güven skoru yine de yüksek miydi?

BLOK 1 • 0:10–1:25 • 75 DK

Veri tabanını kurmak ve doldurmak

18 dk **Veri tabanı ve SQL**	18 dk **Bağlantı ve tablo**	29 dk **Kayıt ekleme**	10 dk **Hatalar**

BLOK 1 • VERI TABANI VE SQL • 18 DK

Neden dosya değil de veri tabanı?

Benzetme: kütüphane kataloğu

Kitapları rastgele bir kutuya atmak yerine (metin dosyası) her kitap için aynı bilgileri taşıyan kartlar düzenli çekmecelerde durur. “1960’tan önce yazılmış romanlar” sorusunun cevabı saniyeler sürer.

İhtiyaç	Metin/JSON	Veri tabanı
Binlerce kayıt	yavaşlar	hızlı
Belirli kayıtları bulmak	tümünü oku, döngü	tek sorgu
Bir kaydı değiştirmek	tüm dosyayı yeniden yaz	tek komut
Aynı anda çok kullanıcı	bozulma riski	kontrollü
Veri kuralları	yok	NOT NULL, UNIQUE…

BLOK 1 • VERI TABANI VE SQL • 18 DK

Tablo, satır, sütun, birincil anahtar

id	ad	yazar	yil	puan
1	Tutunamayanlar	Oğuz Atay	1972	5
2	İnce Memed	Yaşar Kemal	1955	4
3	Saatleri Ayarlama Enstitüsü	Ahmet Hamdi Tanpınar	1961	5
4	Kürk Mantolu Madonna	Sabahattin Ali	1943	4

TABLO

kitaplar

Aynı türden kayıtların listesi; Excel'deki bir sayfa gibi.

SATIR (kayıt)

bir kitap

Bir kitabın bütün bilgileri.

SÜTUN (alan)

yazar

Her kayıtta aynı türden bir bilgi; bir veri tipi var.

BİRİNCİL ANAHTAR

id

Her satırı benzersiz tanımlayan numara; iki kitap aynı ada sahip olabilir.

BLOK 1 • VERI TABANI VE SQL • 18 DK

SQL: veri tabanıyla konuşma dili

```sql
CREATE TABLE kitaplar (ad TEXT, yazar TEXT);

INSERT INTO kitaplar (ad, yazar)
VALUES ('İnce Memed', 'Yaşar Kemal');

SELECT ad, yazar FROM kitaplar;
```

CRUD	SQL komutu	Ders
Create (oluştur)	INSERT	14
Read (oku)	SELECT	14
Update (güncelle)	UPDATE	15
Delete (sil)	DELETE	15

BLOK 1 • VERI TABANI VE SQL • 18 DK

SQLite: tek dosyalık veri tabanı

	SQLite	PostgreSQL / MySQL
Kurulum	yok, Python ile gelir	sunucu kurulur
Veri	tek bir .db dosyası	sunucu klasörleri
Kullanıcı	bir uygulama	çok kullanıcı, ağ üzerinden
Nerede?	telefonlar, tarayıcılar, masaüstü	web siteleri, şirket sistemleri

Benzetme

SQLite bir defter: çantanızda taşırsınız. Sunucu veri tabanı bir kütüphane binası: herkes gelip kullanır ama bir görevli (sunucu) gerekir.

BLOK 1 • BAĞLANTI VE TABLO • 18 DK

connect, cursor, execute, close

```python
import sqlite3

baglanti = sqlite3.connect("kutuphane.db")  # dosya yoksa oluşturur
imlec = baglanti.cursor()
imlec.execute("SELECT sqlite_version()")
print("SQLite sürümü:", imlec.fetchone()[0])
baglanti.close()
```

```
ÇIKTI
SQLite sürümü: 3.45.1
```

connect

bağlantı

Dosyayı açar (yoksa oluşturur).

cursor

imleç

Komutları gönderen, sonuçları getiren araç.

execute

çalıştır

Bir SQL komutunu gönderir.

close

kapat

İş bitince bağlantıyı kapatır.

CREATE TABLE: tablonun kalıbını çizmek

```
import sqlite3

baglanti = sqlite3.connect("kutuphane.db")
imlec = baglanti.cursor()
imlec.execute("""
CREATE TABLE IF NOT EXISTS kitaplar (
    id    INTEGER PRIMARY KEY AUTOINCREMENT,
    ad    TEXT NOT NULL,
    yazar TEXT NOT NULL,
    yil   INTEGER,
    puan  INTEGER DEFAULT 3
)""")
baglanti.commit()
print("Tablo hazır.")
baglanti.close()
```

BLOK 1 • BAĞLANTI VE TABLO • 18 DK

SQLite veri tipleri ve sütun kuralları

SQLite tipi	Python	Örnek
INTEGER	int	1972
REAL	float	245.90
TEXT	str	'İnce Memed'
BLOB	bytes	resim, dosya
NULL	None	boş değer

Kural	Anlamı
PRIMARY KEY	benzersiz kimlik
AUTOINCREMENT	otomatik artan sayı
NOT NULL	boş bırakılamaz
UNIQUE	tekrar edemez
DEFAULT x	verilmezse x

BLOK 1 • KAYIT EKLEME • 29 DK

INSERT INTO: tabloya kayıt eklemek

```
import sqlite3

baglanti = sqlite3.connect("kutuphane.db")
imlec = baglanti.cursor()
imlec.execute("INSERT INTO kitaplar (ad, yazar, yil, puan) VALUES (?, ?, ?, ?)",
              ("Tutunamayanlar", "Oğuz Atay", 1972, 5))
baglanti.commit()
print("Eklenen kaydın id'si:", imlec.lastrowid)
baglanti.close()
```

```
ÇIKTI (ilk çalıştırmada)
Eklenen kaydın id'si: 1
```

? yer tutucuları sırayla demetteki değerlerle doldurulur. id'yi biz vermedik: SQLite verdi.

BLOK 1 • KAYIT EKLEME • 29 DK

Neden f-string değil de ?

YANLIŞ: değeri SQL metnine gömmek

```
ad = "Ali'nin Hikâyesi"
baglanti.execute(f"INSERT INTO kitaplar VALUES ('{ad}', '{yazar}')")
```

```
HATA
sqlite3.OperationalError: near "nin": syntax error
```

DOĞRU: ? yer tutucusu

```
baglanti.execute("INSERT INTO kitaplar VALUES (?, ?)", (ad, yazar))
```

```
ÇIKTI
[("Ali'nin Hikâyesi", 'Ayşe')]
```

SQL enjeksiyonu

Kullanıcının yazdığı metin SQL'in parçası olursa, kötü niyetli biri tabloyu silen bir komut bile "yazabilir". ? kullanınca değer her zaman veri olarak kalır.

BLOK 1 • KAYIT EKLEME • 29 DK

executemany: listedeki bütün kayıtları eklemek

```
KITAPLAR = [
    ("Tutunamayanlar", "Oğuz Atay", 1972, 5),
    ("İnce Memed", "Yaşar Kemal", 1955, 4),
    ("Saatleri Ayarlama Enstitüsü", "Ahmet Hamdi Tanpınar", 1961, 5),
    ("Kürk Mantolu Madonna", "Sabahattin Ali", 1943, 4),
    ("Puslu Kıtalar Atlası", "İhsan Oktay Anar", 1995, 4),
    ("Kuyucaklı Yusuf", "Sabahattin Ali", 1937, 3),
    ("Benim Adım Kırmızı", "Orhan Pamuk", 1998, 3),
]
```

```
imlec.executemany(
    "INSERT INTO kitaplar (ad, yazar, yil, puan)"
    " VALUES (?, ?, ?, ?)",
    KITAPLAR[1:])
baglanti.commit()
print(imlec.rowcount, "kitap eklendi.")
```

```
ÇIKTI
6 kitap eklendi.
```

BLOK 1 • KAYIT EKLEME • 29 DK

commit: değişiklikleri kalıcı yapmak

Benzetme: belgeyi kaydetmek

Word'de yazdıklarınız "Kaydet"e basana kadar yalnızca ekranda durur. INSERT yazmak gibidir; commit kaydetmektir. commit'siz kapatırsanız eklenenler kaybolur.

```
with baglanti:  # hatasız biterse commit
    baglanti.execute(
        "INSERT INTO kitaplar (ad, yazar, yil)"
        " VALUES (?, ?, ?)",
        ("Semaver", "Sait Faik Abasıyanık", 1936))
```

```
ÇIKTI (ilk çalıştırmada)
Toplam kitap: 8
```

BLOK 1 • SIK YAPILAN HATALAR

Tablo ve INSERT hataları

Hatalı kod	Hata mesajı	Çözüm
`CREATE TABLE kitaplar … (2. kez)`	`sqlite3.OperationalError: table kitaplar already exists`	IF NOT EXISTS
`SELECT * FROM kitap`	`sqlite3.OperationalError: no such table: kitap`	tablo adını / dosya adını kontrol
`INSERT … (ad) VALUES (?) # yazar yok`	`sqlite3.IntegrityError: NOT NULL constraint failed: kitaplar.yazar`	zorunlu sütunu verin
`… VALUES (?)", ("Tutunamayanlar"))`	`sqlite3.ProgrammingError: Incorrect number of bindings supplied. The current statement uses 1, and there are 14 supplied.`	tek elemanlı demet: ("…",)

1:25–1:40 • 15 DK

Mola

Moladan sonra: veri tabanına koyduk, peki nasıl geri alacağız?

BLOK 2 • 1:40–2:55 • 75 DK

Veriyi okumak ve kullanmak

26 dk **SELECT**	11 dk **DB Browser**	18 dk **Uygulamalar**	20 dk **Alıştırmalar**

BLOK 2 • SELECT • 26 DK

SELECT: kayıtları okumak

```
imlec.execute("SELECT * FROM kitaplar")
satirlar = imlec.fetchall()
print(type(satirlar), len(satirlar))
for satir in satirlar[:3]:
    print(satir)
```

```
ÇIKTI
<class 'list'> 7
(1, 'Tutunamayanlar', 'Oğuz Atay', 1972, 5)
(2, 'İnce Memed', 'Yaşar Kemal', 1955, 4)
(3, 'Saatleri Ayarlama Enstitüsü', 'Ahmet Hamdi Tanpınar', 1961, 5)
```

fetchall() → demetlerden oluşan bir liste. Her demet bir satır: (id, ad, yazar, yil, puan).

BLOK 2 • SELECT • 26 DK

İstediğin sütunlar, istediğin sırayla

```
imlec.execute("SELECT ad, yil FROM kitaplar ORDER BY yil")
for ad, yil in imlec.fetchall():
    print(yil, ad)
```

Tersten sıralama: **ORDER BY yil DESC** • İki ölçüt: **ORDER BY puan DESC, ad**

```
ÇIKTI
1937 Kuyucaklı Yusuf
1943 Kürk Mantolu Madonna
1955 İnce Memed
1961 Saatleri Ayarlama Enstitüsü
1972 Tutunamayanlar
1995 Puslu Kıtalar Atlası
1998 Benim Adım Kırmızı
```

BLOK 2 • SELECT • 26 DK

LIMIT, fetchone ve toplama fonksiyonları

```
imlec.execute("SELECT ad, puan FROM kitaplar ORDER BY puan DESC, yil LIMIT 3")
print("İlk 3:", imlec.fetchall())

imlec.execute("SELECT COUNT(*), AVG(puan), MIN(yil), MAX(yil) FROM kitaplar")
sayi, ort, en_eski, en_yeni = imlec.fetchone()
print(f"{sayi} kitap · ortalama puan {ort:.2f} · {en_eski}-{en_yeni}")
```

```
ÇIKTI
İlk 3: [('Saatleri Ayarlama Enstitüsü', 5), ('Tutunamayanlar', 5), ('Kürk Mantolu Madonna', 4)]
7 kitap · ortalama puan 4.00 · 1937-1998
```

Fonksiyon	COUNT(*)	SUM(s)	AVG(s)	MIN(s) / MAX(s)
Anlamı	satır sayısı	toplam	ortalama	en küçük / en büyük

BLOK 2 • SELECT • 26 DK

sqlite3.Row: sütuna adıyla ulaşmak

```
baglanti.row_factory = sqlite3.Row
for kitap in baglanti.execute("SELECT ad, yazar FROM kitaplar ORDER BY yil LIMIT 3"):
    print(kitap["ad"], "-", kitap["yazar"])
```

```
ÇIKTI
Kuyucaklı Yusuf - Sabahattin Ali
Kürk Mantolu Madonna - Sabahattin Ali
İnce Memed - Yaşar Kemal
```

satir[2] yerine **kitap["yazar"]**: tabloya sütun eklense bile kod bozulmaz, okuması kolay.

BLOK 2 • SELECT • SIK YAPILAN HATALAR

SELECT ve bağlantı hataları

Hatalı kod	Hata mesajı	Çözüm
`SELECT yazar_adi FROM kitaplar`	`sqlite3.OperationalError: no such column: yazar_adi`	sütun adını kontrol
`SELEC * FROM kitaplar`	`sqlite3.OperationalError: near "SELEC": syntax error`	yazımı düzeltin
`baglanti.close() baglanti.execute(…)`	`sqlite3.ProgrammingError: Cannot operate on a closed database.`	close en sonda
`imlec.fetchone()[0] # tablo boş`	`TypeError: 'NoneType' object is not subscriptable`	None kontrolü

BLOK 2 • DB BROWSER • 11 DK

DB Browser for SQLite ile görsel inceleme

Sekme / düğme	Ne işe yarar?
`Open Database`	kutuphane.db dosyasını açar (Ctrl+O)
`Database Structure`	tablolar, sütunlar, tipler ve kurallar
`Browse Data`	satırları tablo halinde görür, elle düzenler
`Execute SQL`	SQL komutu yazıp çalıştırır (Ctrl+Enter)
`Write Changes`	elle yapılan değişiklikleri kaydeder (= commit)
`Close Database`	dosyayı serbest bırakır

Önce Python programını çalıştırın, sonra dosyayı açın. Yeni veriyi görmek için **Browse Data**'da yenileyin (F5).

BLOK 2 • DB BROWSER • 11 DK

İki program, bir dosya: “database is locked”

Python’da görülen hata

```
sqlite3.OperationalError: database is locked
```

Neden?

DB Browser’da kaydedilmemiş (Write Changes yapılmamış) bir değişiklik varken dosya kilitlenir. Python yazmak isteyince bekler, sonra bu hatayı verir.

Durum	Yapılacak
DB Browser’da elle değişiklik yaptınız	Write Changes ya da Revert Changes
Execute SQL ile INSERT çalıştırdınız	Write Changes (Ctrl+S)
Emin değilseniz	Close Database, sonra Python’u çalıştırın

BLOK 2 • UYGULAMALAR • 18 DK

Uygulama 1: menülü kitaplık programı

```
ders14 · Uygulama 1 (kısaltılmış)
def kitap_ekle(ad, yazar, yil, puan):
    with baglanti:
        baglanti.execute("INSERT INTO kitaplar (ad, yazar, yil, puan)"
                         " VALUES (?, ?, ?, ?)", (ad, yazar, yil, puan))

def kitaplari_listele():
    satirlar = baglanti.execute("SELECT id, ad, yazar, yil, puan"
                                " FROM kitaplar ORDER BY yil").fetchall()
    for no, ad, yazar, yil, puan in satirlar:
        print(f"{no:>3}. {ad} — {yazar} ({yil}) · puan {puan}/5")

while True:
    secim = input("Seçim: ").strip()
    if secim == "1": ...       # bilgileri al, kitap_ekle
    elif secim == "2": kitaplari_listele()
    elif secim == "0": break
```

KATMANLAR

Veri ↔ arayüz

SQL yalnızca fonksiyonların içinde; menü döngüsü SQL bilmez.

KALICI

kitaplik.db

Program kapanıp açılınca kitaplar yerinde.

BLOK 2 • UYGULAMALAR • 18 DK

Uygulama 1'i çalıştırınca

```
ÇIKTI (1. bölüm)

1) Ekle  2) Listele  3) İstatistik  0) Çıkış
Seçim: 1
Kitap adı: Kürk Mantolu Madonna
Yazar: Sabahattin Ali
Yıl: 1943
Puan (1-5): 4
Eklendi.

1) Ekle  2) Listele  3) İstatistik  0) Çıkış
Seçim: 1
Kitap adı: İnce Memed
Yazar: Yaşar Kemal
Yıl: 1955
Puan (1-5): 5
Eklendi.
```

```
ÇIKTI (devamı)

1) Ekle  2) Listele  3) İstatistik  0) Çıkış
Seçim: 2
  1. Kürk Mantolu Madonna — Sabahattin Ali (1943) · puan 4/5
  2. İnce Memed — Yaşar Kemal (1955) · puan 5/5

1) Ekle  2) Listele  3) İstatistik  0) Çıkış
Seçim: 3
2 kitap, ortalama puan 4.5

1) Ekle  2) Listele  3) İstatistik  0) Çıkış
Seçim: 0
```

BLOK 2 • UYGULAMALAR • 18 DK

Uygulama 2: harcama defteri artık veri tabanında

```
baglanti = sqlite3.connect("harcamalar.db")
with baglanti:
    baglanti.execute("""CREATE TABLE IF NOT EXISTS harcamalar (
        id INTEGER PRIMARY KEY AUTOINCREMENT, tarih TEXT NOT NULL,
        aciklama TEXT NOT NULL, tutar REAL NOT NULL)""")
    baglanti.executemany("INSERT INTO harcamalar (tarih, aciklama, tutar)"
                         " VALUES (?, ?, ?)", HARCAMALAR)
adet, toplam = baglanti.execute(
    "SELECT COUNT(*), SUM(tutar) FROM harcamalar").fetchone()
print(f"{adet} harcama, toplam {toplam:.2f} TL")
for tarih, aciklama, tutar in baglanti.execute("SELECT tarih, aciklama,"
        " tutar FROM harcamalar ORDER BY tutar DESC"):
    print(f"{tarih}  {aciklama:<7}{tutar:>8.2f} TL")
```

```
ÇIKTI (ilk çalıştırmada)
3 harcama, toplam 486.40 TL
2026-10-20  Market   245.90 TL
2026-10-23  Kitap    180.50 TL
2026-10-21  Ulaşım    60.00 TL
```

Ders 10'daki **tarih;açıklama;tutar** dosyasının yerini bir tablo aldı; toplamı SQL hesapladı.

BLOK 2 • SINIF IÇI ALIŞTIRMALAR • 15 DK

Alıştırma 1, 2 ve 3

KOLAY • 4 dk

1 • Film tablosu

filmler.db içinde filmler tablosu (id, ad, yonetmen, yil) oluşturun; üç film ekleyip hepsini yıla göre listeleyin.

ORTA • 5 dk

2 • En iyi üç kitap

KITAPLAR listesini :memory: veri tabanına ekleyin; en yüksek puanlı 3 kitabı ve kitapların ortalama yayın yılını yazdırın.

ZOR • 6 dk

3 • Güvenli ekleme

kitap_ekle(ad, yazar, yil, puan) yazın: puan 1–5 dışındaysa ValueError fırlatsın, eklenen kaydın id'sini döndürsün. try/except ile deneyin.

BLOK 2 • ALIŞTIRMA ÇÖZÜMLERI

Çözüm 1: film tablosu

```
baglanti = sqlite3.connect("filmler.db")
with baglanti:
    baglanti.execute("""CREATE TABLE IF NOT EXISTS filmler (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        ad TEXT NOT NULL, yonetmen TEXT NOT NULL, yil INTEGER)""")
    baglanti.executemany("INSERT INTO filmler (ad, yonetmen, yil)"
                         " VALUES (?, ?, ?)", FILMLER)
for ad, yonetmen, yil in baglanti.execute(
        "SELECT ad, yonetmen, yil FROM filmler ORDER BY yil"):
    print(f"{yil} · {ad} ({yonetmen})")
```

```
ÇIKTI (ilk çalıştırmada)
1996 · Eşkıya (Yavuz Turgul)
2005 · Babam ve Oğlum (Çağan Irmak)
2014 · Kış Uykusu (Nuri Bilge Ceylan)
```

Çözüm 2: en iyi üç kitap

```
baglanti = sqlite3.connect(":memory:")
baglanti.execute("CREATE TABLE kitaplar"
                 " (ad TEXT, yazar TEXT, yil INTEGER, puan INTEGER)")
baglanti.executemany("INSERT INTO kitaplar VALUES (?, ?, ?, ?)", KITAPLAR)
for ad, puan in baglanti.execute("SELECT ad, puan FROM kitaplar"
                                 " ORDER BY puan DESC, ad LIMIT 3"):
    print(f"  {ad} ({puan})")
ort_yil = baglanti.execute(
    "SELECT AVG(yil) FROM kitaplar").fetchone()[0]
print(f"Ortalama yayın yılı: {ort_yil:.0f}")
```

```
ÇIKTI
En yüksek puanlı 3 kitap:
  Saatleri Ayarlama Enstitüsü (5)
  Tutunamayanlar (5)
  Kürk Mantolu Madonna (4)
Ortalama yayın yılı: 1966
```

Çözüm 3: güvenli ekleme fonksiyonu

```
def kitap_ekle(baglanti, ad, yazar, yil, puan):
    if not 1 ⩽ puan ⩽ 5:
        raise ValueError("Puan 1 ile 5 arasında olmalı.")
    with baglanti:
        imlec = baglanti.execute(
            "INSERT INTO kitaplar (ad, yazar, yil, puan)"
            " VALUES (?, ?, ?, ?)", (ad, yazar, yil, puan))
    return imlec.lastrowid

for kitap in [("Tutunamayanlar", "Oğuz Atay", 1972, 5),
              ("Deneme", "Yazar", 2020, 9)]:
    try:
        print("Eklendi, id:", kitap_ekle(baglanti, *kitap))
    except ValueError as hata:
        print("Eklenmedi:", hata)
```

```
ÇIKTI
Eklendi, id: 1
Eklenmedi: Puan 1 ile 5 arasında
  olmalı.
```

KAPANIŞ • 5 DK

Bugün ne öğrendik?

- Veri tabanı = tablolar; tablo = satırlar × sütunlar; her satırın bir **birincil anahtarı** var.
- **sqlite3.connect** → **execute** → **commit** → **close**; **with baglanti:** otomatik commit.
- **CREATE TABLE IF NOT EXISTS**; tipler INTEGER, REAL, TEXT; kurallar NOT NULL, DEFAULT, UNIQUE.
- **INSERT** her zaman **?** yer tutucularıyla; f-string ile SQL yok.
- **SELECT … ORDER BY … LIMIT**, **COUNT/SUM/AVG/MIN/MAX**; **fetchall** liste, **fetchone** tek satır.

KAPANIŞ • ÖDEV • TESLIM: BIR SONRAKI DERS

Ödev

1 • ZORUNLU

Film arşivi programı

Uygulama 1'i örnek alarak filmler.db için menülü bir program yazın: film ekle, yıla göre listele, film sayısı ve ortalama yıl.

2 • ZORUNLU

DB Browser ile

filmler.db'yi DB Browser'da açın; Browse Data'da iki film ekleyip Write Changes yapın. Programınızda listeleyip göründüğünü kontrol edin; ekran görüntüsü alın.

3 • BONUS

Analizleri kaydet

Ders 13 yorum analizi uygulamasında her analizi (tarih, yorum, duygu, güven) bir SQLite tablosuna kaydedin; geçmiş sayfa yenilense de kalsın.

Teslim: ders14_odev.py ya da .ipynb. Teslim yöntemi: [teslim yöntemi]

KAPANIŞ • GELECEK DERS: HAFTA 8 • GÜN 1

SQLite 2: filtreleme, güncelleme, silme

Bugün tablonun tamamını okuduk. Gelecek ders yalnızca istediğimiz kayıtlar ve kayıtları değiştirmek:

- WHERE ile koşul: =, <, >, BETWEEN, IN
- LIKE ile metin arama (“%Ali%”)
- AND / OR / NOT
- UPDATE ve DELETE • mini proje: kitaplık yönetimi

Önizleme

```
imlec.execute("""SELECT ad, yil FROM kitaplar
    WHERE yil < 1960 AND puan ⩾ 4
    ORDER BY yil""")
print(imlec.fetchall())
```

```
[('Kürk Mantolu Madonna', 1943), ('İnce Memed', 1955)]
```

KAYNAKLAR

Bu dersin konuları resmi belgelerde

Ne için?	Adres
sqlite3 modülü (Python belgeleri)	docs.python.org/3/library/sqlite3.html
SQLite SQL dili	sqlite.org/lang.html
SQLite veri tipleri	sqlite.org/datatype3.html
DB Browser for SQLite	sqlitebrowser.org
SQL alıştırmaları (etkileşimli)	sqlbolt.com